Ders 11 Genişletilmiş Notu: Kodlama, Yazılım Kalite Faktörleri, Test Aşamaları, Yeni Sisteme Geçiş Yöntemleri ve Bakım

Ders 11 Genişletilmiş Notu: Kodlama, Yazılım Kalite Faktörleri, Test Aşamaları, Yeni Sisteme Geçiş Yöntemleri ve Bakım

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Dersin Planı ve Kodlamanın Süreçteki Yeri
- 2. Kodlama Standartları ve İyi Kodun Özellikleri
- 3. Programlama Dili Seçimi
- 4. Yazılım Kalitesi ve Kalite Faktörleri
- 5. Kalite Nasıl Sağlanır ve Ölçülür?
- 6. Test Aşaması ve Test Türleri
- 7. Test Adımları: Test Spirali
- 8. Yeni Sisteme Geçiş İçin Hazırlık
- 9. Yeni Sisteme Geçiş Yöntemleri
- 10. Bakım
- 11. Döngünün Yeniden Başlaması ve Günümüz Yaklaşımları

1. Dersin Planı ve Kodlamanın Süreçteki Yeri

Bu ders, teorik anlatımın son dersidir. Hoca dört başlığı üst seviyede ele alacağını söyledi. Önce kodlamaya kısaca değinecek ve yazılım kalitesinin bizi nasıl etkilediğini, başımıza neler açtığını anlatacak. Sonra kaliteyi sağlamak ve “doğru şeyi yapıyor muyuz?” diye bakmak için kullanılan temel test tekniklerini, çok detaya girmeden görecek. Ardından sondan bir önceki adım gelecek: Bir problemi ele aldık, çözüm ürettik, yeni bir bilgi sistemi geliştirdik. Bu sistem nasıl kullanılmaya başlayacak, mevcut sistemden yeni sisteme nasıl geçilecek? Son olarak bir bilgi sisteminin yaşamındaki en uzun süreç olan bakımın ne olduğu, ne gibi bakımların neden yapıldığı işlenecek.

1.1 Şelale Modelinde Kodlamaya Gelene Kadar

Kodlama, yani gerçekleştirme aşamasının yerini görmek için hoca şelale modelindeki yolu hatırlattı. Problemi aldık ve ön incelemesini yaptık. Fizibilite çalışması gerçekleştirdik: Bu projeyi yapabiliyor muyuz, yapmak nelere mal olur diye baktık. Problemi anladık, modelledik ve sistem analizini alt parçalarıyla birlikte yaptık. Sonra bir çözüm önerisi, bir çözüm taslağı geliştirdik. Buna da tasarım dedik.

Dersin adı Sistem Analizi ve Tasarımı olduğu için “tasarımla iş bitti, burada duralım” da denebilirdi. Ama bir bilgi sisteminin oluşması için bunların gerçekleştirilmesi gerekir. Şelale modelinde bütün diyagramlar, algoritmalar, davranış akışları ve sözde kod (hocanın ifadesiyle “pseudo code”) anlatımları tamamlanınca “hadi bakalım oturalım kodlayalım” aşamasına geçilir.

1.2 Tasarım Olmadan Kodlamak

Hoca, günlük hayatta çoğu öğrencinin böyle yapmadığını, ders projesinde bile çoğunluğun doğrudan koddan başlamış olabileceğini söyledi. Oysa özellikle büyük projelerde olması gereken, adım adım ilerlemektir: Önce problemi anlamak ve tanımlamak, sonra çözüm önerilerini ortaya koymak, en son gerçekleştirmeyi yapmak.

Bu sıra, işin tek seferde yapılması gerektiği anlamına gelmez. Daha önce anlatılan spiral model gibi yaklaşımlarla iş küçük parçalar hâlinde, iteratif olarak da yapılabilir. Günümüzün çevik süreçlerinde esas olan zaten budur: Problem küçük parçalar eklenerek çözülür, sürekli yazılım üretilir, yazılım sürekli devreye alınır, yeni parçalarla güncel tutulur. Ama orada bile küçük bir analiz ve küçük bir tasarımın ardından gerçekleştirme gelir. Bu küçük parçalar birbirini takip eder.

> **Hocanın vurgusu:** Bir tasarım olmadan, en azından bir tasarım fikri olmadan gerçekleştirmeye geçmek çoğu zaman bizi kötü ya da yanlış bir ürüne taşır. Bu yüzden kodlamayı tasarımdan sonra yapmak daha doğrudur.

2. Kodlama Standartları ve İyi Kodun Özellikleri

Hoca, projenin bir ortamda nasıl açılacağı, dosyaların ve kütüphanelerin nasıl bağlanacağı gibi ayrıntıları anlatmayacağını söyledi. Bunları öğrenciler elini kirleterek ya buldu ya da buluyor. Asıl söylemek istediği başka bir şeydi.

> **Hocanın vurgusu:** En önemli şey, özellikle büyük bir bilgi sistemi geliştirme projesinde çalışıyorsanız, kodlamada standartlara uymaya özen göstermektir.

Bunun karşılığı “coding conventions” denen, kullandığınız dile özgü önerilerdir. O dile özgü kodlama desenlerini ve standartlarını kullanmak esastır. Bazı diller bazı desenleri daha çok tercih eder ve ister. Bu kuralların amacı aşağıdaki özellikleri sağlamaktır.

2.1 Kolay Okunabilirlik

Bir koda baktığınızda kodun “elinin yüzünün düzgün olması” gerekir. Kod bloklarının, karar dallanmalarının ve döngü bloklarının net olması, gerekli girintilerin, bloklama işaretlerinin ve açıklamaların bulunması beklenir. Baktığımızda kodun ne yaptığını kolayca okuyabilmek, bir kodun tercih sebebidir.

2.2 Kolay Düzenlenebilirlik

Kodun düzeni de önemlidir. Prototipler önce mi verilecek, belirli fonksiyonlar altta mı üstte mi yazılacak, ana fonksiyonlar nerede duracak, değişken tanımları nasıl yapılacak? Bu desenler dilden dile değişir. Ama uyulduğunda kod, daha sonraki güncelleme ve değişikliklerde rahat düzenlenir.

2.3 Test Edilebilirlik

Kod, test standartlarına uygun ve test edilebilir olarak yazılmalıdır. Java gibi dillerde birim testi için standart bir kütüphane kullandığınızda neler yapmanız gerektiği bellidir. Başka dillerde başka kütüphaneler için de bellidir.

> **Ders sonrası not:** Altyazıda Java için birim testi kütüphanesi “C-Unit” olarak geçiyor. Bu büyük olasılıkla bir altyazı hatasıdır. Java’nın yaygın birim testi kütüphanesinin adı JUnit’tir.

Hoca, öğrencilerin şu anda buna belki uzak olabileceğini, önemini özellikle Yazılım Mühendisliği dersiyle birlikte daha fazla göreceklerini söyledi. Bu ders, konuyla bir tanışma safhasıdır.

2.4 Kolay Değiştirilebilirlik

Kolay değiştirilebilirlik, düzenlenebilirlikten çok farklı değildir. Yapısal değişikliklerin, ilavelerin ya da çıkarmaların kod üzerinde kolayca yapılabilmesi anlamına gelir.

2.5 Açıklama Satırları

> **Hocanın vurgusu:** Yazdığınız her kod için mutlaka açıklama satırları kullanın.

Bunu düzgün yaparsanız kodun dokümantasyonunu da otomatik olarak oluşturursunuz. Pek çok programlama dilinde, yorum satırları istenen biçimde yazıldığında API dokümanı bu satırlardan otomatik üretilebilir.

Hoca, yazılan bir satır kod için neredeyse iki satır açıklama yazmak gerektiğini söyledi. Yazmazsanız kodunuz kolay okunmaz. Başkaları düzenlerken zorlanır, siz bile kendi kodunuzu düzenlerken şaşırabilirsiniz. Bir hafta önce yazdığınız koda dönüp baktığınızda yeterince yorum yoksa bakıp kalırsınız: "Ben burada ne yapmışım? Bu if'i niye böyle yazmışım? Bu döngüde niye böyle yapmışız? Bu nereden geliyor, bunu nerede tanımlamıştık?"

> **Ders sonrası not:** Hoca "bir satır koda neredeyse iki satır açıklama" dedi. Bu oran sabit bir kural olarak alınmamalıdır. Hocanın asıl vurgusu, açıklamasız kodun zamanla okunamaz hâle gelmesidir. Güncel uygulamada anlamlı isimli, kendini açıklayan kod ile kodun "neden"ini anlatan yorumlar birlikte önerilir.

2.6 Anlamlı İsimlendirme

Sınıf ve nesne isimlerinden dizilere, değişkenlere ve iteratörlere kadar kullandığınız her veri yapısının ismi anlamlı olmalıdır. Değişkenlere A, B, C, D diye isim verirseniz bir hafta sonra açtığınızda "A neyi tutuyordu, B neyi tutuyordu?" diye kalırsınız.

Hoca bunun tersi bir hataya da dikkat çekti: Çok jenerik isimler de kullanılmamalıdır. "Toplam" ya da "ortalama" yeterli değildir. Neyin toplamı ya da neyin ortalaması olduğunu söyleyen isimler tercih edilmelidir.

3. Programlama Dili Seçimi

Kodlama için bir programlama diline ve programlama ortamına ihtiyaç vardır. Hoca bunların aslında daha önce seçildiğini hatırlattı: Gereksinimleri karşılayacak biçimde fizibilitede seçilmiş, ön tasarım aşamasında kesinleştirilmişti. Bir bilgi sistemi için dil seçerken dikkat edilecek noktalar şunlardır:

- **Dilin genel uygulama alanı:** Bazı diller daha matematik ağırlıklı, bazıları işlem (transaction) ağırlıklı, bazıları karakter dizisi (string) işlemleri ağırlıklı, bazıları tamamen genel amaçlıdır. Bazıları bilimsel program geliştirmeye daha uygundur. Sizin alanınıza yakın olan dil, daha kolay gerçekleştirme yapmanızı sağlar.
- **Algoritmaların ve veri yapılarının karmaşıklığı:** Bazı dillerde bazı veri yapıları hazırdır, bazılarını sizin yazmanız gerekir. Bu da tercih sebebi olabilir.
- **Yazılımın kullanılacağı ortam:** Yazılım gömülü bir ortamda çalışacak olabilir ve dil o platformu desteklemiyor, derleyicisi yok olabilir. Ya da dil, o ortamdaki kötü performansıyla bilinebilir.
- **Uygulamanın çalışma koşulları:** Dokunmatik girdi almanız gerekiyorsa ve dil, mobil gibi ön yüzlerle rahat konuşamıyorsa bu sizi zorlar. Dokunmatikten veri akışı için hazır destek sağlıyorsa tercih sebebi olur.
- **Personelin bilgi düzeyi:** Kodu insanlar yazar. Hiç bilinmeyen bir dilde "develenmek" yerine, bilinen bir dilde çok daha hızlı uygulama geliştirilir. Bu yüzden dil ekibe göre de seçilebilir.
- **Lisans ve işletme bedelleri:** Dilin çalıştığı uygulama sunucusunun (application server) belli bir lisans bedeli olabilir. Bu bedel kira gibi periyodik olarak da ödenebilir. Bunlar yatırımı etkiler, bu yüzden tartışılmalıdır.
- **Müşterinin özel koşulları:** Müşteri, "Bütün sistemlerimiz Java, bu yüzden .NET istemiyorum" diyebilir. Ya da "Python kullanıyoruz, bütün kütüphanelerimizi orada yazdık, bu şekilde devam etmek istiyoruz, R kullanamazsınız" diyebilir.

Özetle hangi dil size en hızlı, en ucuz, en temiz ve en kolay biçimde hizmet ediyorsa onu seçmek daha doğrudur. Bu, daha kaliteli bir bilgi sistemi ortaya koymanızı sağlar.

> **Hocanın vurgusu:** Her dili her noktasına kadar bilemezsiniz. Ama diller sizin için birer araçtır. Amaca uygun aracı seçip kullanma sorumluluğu ve yeteneği, mühendis

olarak tamamen sizin omuzlarınızdadır.

4. Yazılım Kalitesi ve Kalite Faktörleri

Hoca bu başlığa, bir bilgi sisteminde belki de bizi en çok zorlayan, başımıza bela olan noktalardan biri diyerek girdi.

4.1 Kalitenin Tanımı

Yazılım için kalite, işlevsel gereksinimlere, geliştirme standartlarına ve beklenen tüm özelliklere tamamen uyan bir yazılımın geliştirilmesi demektir. Tanımda işlevsel gereksinimler ve standartlar açıkça yazıyor, ama yazmayan önemli bir şey var. Yazılımın kalite faktörlerinin her biri, bir bilgi sistemindeki işlevsel olmayan gereksinimlerden doğar ya da onları doğrudan etkiler.

4.2 Faktörlerin İşlevsel Olmayan Gereksinimlerle İlişkisi

Hoca bunu önce iki faktörle gösterdi. Doğruluk, doğru girdiler verildiğinde doğru sonucun üretilmesidir. Bu, işlevsel bir beklenti olarak da görülebilir. Güvenilirlik (reliability) için ise şu örneği verdi: Herhangi bir T anında 10.000 kullanıcı veri girişi yaparken sistemin tepki süresinin makul sınırı nedir, 5 milisaniye mi, 10 milisaniye mi, 100 milisaniye mi? Sistem bu sınırın altında kalmalıdır. Ya da belirli bir yük veya akış hızı altında bile güvenilir biçimde cevap üretmeye devam edebilmelidir. Bu, işlevsel bir gereksinim olarak ifade edilemez. İşlevsel olmayan bir gereksinimdir ve doğrudan kaliteyle ilgilidir.

> **Ders sonrası not:** Hoca güvenilirliği tepki süresinin yük altında makul sınırda kalmasıyla örnekledi. Tepki süresi daha çok verimlilik ve performansla ilgilidir. Güvenilirlik, sistemin belirli koşullarda belirli bir süre boyunca hatasız çalışmasıdır. Hocanın "yük altında cevap üretmeye devam edebilmesi" ifadesi bu tanıma daha yakındır.

Bir kullanıcı şöyle konuşabilir: "İşi yapıyor mu? Yapıyor. Ama bu ne biçim yapıyor, çok yavaş, iki kişi girdi mi tıkanıyor." Hoca buna verimlilik dedi. Kullanışlılık da benzerdir. Her biri üzerine çok uzun konuşulabilir, yazılım kalitesi kendi başına bir dönemlik derstir.

Hoca bu faktörlerin neden "başımıza dert açtığını" ders kaydı örneğiyle anlattı. Ders kaydının senaryosunu yazabilir, sınıflarını ve modüllerini oluşturabilirim. Ama "Aynı anda bağlanan 10.000 kullanıcı için tepki süresi 100 milisaniyeyi geçmemelidir" denirse yazılımın mimarisini çok farklı tasarlamam gerekir. Dağıtık yapılar kurgulamam, servisleri farklı biçimde parçalamam gerekir. Bir anda bütün tasarım değişir. Tasarımın değişmesi, doğal olarak gerçekleştirmenin de değişmesi demektir.

4.3 Verimlilik Örneği

Verimlilik, kaynakların etkin kullanımı olarak özetlenebilir. Hocanın örneğinde bir yazılım, bir kayıt için 1 megabayt yer kaplıyor. Aynı işi yapan alternatif bir bilgi sistemi aynı veriyi yalnızca 100 kilobaytta tutuyor ve aynı hızda tepki veriyor.

- **Verilen:** Birinci sistemde kayıt başına 1 MB, ikinci sistemde 100 KB. Tepki hızı aynı.
- **İşlem (ara adım eklendi):** $1\ \text{MB}/100\ \text{KB} = 1000/100 = 10$. Yani birinci sistem kayıt başına yaklaşık 10 kat yer kaplıyor.
- **Sonuç:** İkinci sistem, disk alanını daha verimli kullanıyor.

Aynı tartışma bellek, işlemci gibi başka kaynaklar için de yapılabilir.

4.4 On Bir Kalite Faktörü

Hoca, temel yazılım kalite faktörlerini sayarken önce yedi diyecek oldu, sonra sayarak 11 olduğunu doğruladı:

- **Doğruluk:** Doğru girdilerle doğru sonucun üretilmesi.
- **Güvenilirlik (reliability):** Belirli yük altında güvenilir biçimde cevap üretmeye devam etmek.

- **Verimlilik:** Kaynakların etkin kullanımı.
- **Güvenlik (security):** Güvenli olmak, ekstra geliştirmeler ve ilaveler yapmak demektir.
- **Kullanışlılık:** Kullanıcı için kolaylık.
- **Hata bulma kolaylığı.**
- **Esneklik.**
- **Test edilebilirlik:** Kolay test edilebilmek.
- **Taşınabilirlik:** Farklı platformlarda kolayca kullanılabilmek.
- **Tekrar kullanılabilirlik:** Yazılım parçalarının başka projelerde ve sistemlerde de kullanılabilmesi.
- **Bağlanabilirlik:** Sistemin başka sistem ve yazılımlarla, standart protokoller de kullanarak ne kadar rahat konuşup entegre olabildiği. Hoca bunu adaptasyon yeteneği olarak da andı.

  Ek bilgi (derste anlatılmadı): Sayılan 11 faktör, büyük olasılıkla McCall'un yazılım kalite modelindeki faktörlere dayanır. Bu modelde "güvenlik" faktörünün karşılığı bütünlük (integrity), "hata bulma kolaylığı" faktörünün karşılığı bakım kolaylığı (maintainability), "bağlanabilirlik" faktörünün karşılığı birlikte çalışabilirliktir (interoperability).

4.5 Kalite Maliyettir

> **Hocanın vurgusu:** Bu faktörlerin hepsi bizim için çok ciddi maliyet demektir. Bir kalite faktöründe istenen belirli bir hedef varsa, analiz, tasarım ve gerçekleştirmenin hepsi bu hedef göz önüne alınarak yapılmalıdır. Bu, bütün proje sürecini çok fazla değiştirebilir.

Bu nedenle kalite beklentileri, daha ön inceleme aşamasında göz ardı edilmeden sorulmalıdır: Bu olacak mı, ne kadar olacak, burada ne bekliyorsunuz? Bütün kalite faktörlerinin üzerinden en başta geçmek ve bunları işlevsel çözümlemeye geçmeden önce kesinleştirmek daha doğru bir yoldur.

5. Kalite Nasıl Sağlanır ve Ölçülür?

Kalite faktörleri için pek çok ilave iş yapmak gerekiyorsa, kalitenin sağlanıp sağlanmadığını nasıl göreceğiz? Hoca bunu özet olarak şöyle anlattı:

- **Planlama:** Daha proje planlanırken kalitenin hangi yöntemle kontrol edileceği ve hangi yazılım kalite kontrol araçlarının kullanılacağı belirlenir.
- **Metrik ve ölçüm:** Metrikler belli olmalı, ölçümler de gerçekten proje sürecinde yapılmalıdır.
- **Durak noktaları:** Durak noktaları (milestone) belirlenir ve bu noktalarda mutlaka gözden geçirme yapılır. Kalite faktörleri ne durumda, neyi ne kadar karşıladım, karşılamak için daha neler yapmam lazım? İş etap etap ilerletilir.
- **Test planı:** Mutlaka bir test planı olmalıdır. Kaynak kodlar ve yazılımlar farklı testlerle sınanmalı, doğrulukları geçerlenmelidir.

5.1 Ürünü Doğru Yapmak ve Doğru Ürünü Yapmak

Projede iki ayrı soru sorulur ve ikisi de test edilmelidir: **Ürünü doğru yapıyor muyuz? Doğru ürünü yaptık mı?** Hoca, ürünün doğru olup olmadığının gereksinimlere bakılarak test edileceğini söyledi. İşlevsel olan ve olmayan gereksinimler karşılanıyorsa doğru ve kaliteli bir sistem geliştirmiş oluruz. Karşılanmıyorsa, karşılanmadığı oranda yanlış ya da kalitesiz bir sistem ortaya koymuş oluruz.

> **Ek bilgi (derste anlatılmadı):** Bu iki soru genellikle iki terimle anılır. "Ürünü doğru yapıyor muyuz?" doğrulama (verification), "Doğru ürünü yapıyor muyuz?" geçerleme (validation) olarak adlandırılır. İlki ürünün tanımlanan şartnameye uygunluğuna, ikincisi kullanıcının gerçek ihtiyacını karşılamasına bakar.

5.2 Kalite Planları

Bu işlerin her biri için bir plan bulunur:

- **Yazılım inceleme planı.**
- **Kaynak program sınama planı:** Kabaca test planı ya da yazılım test planı da denebilir.
- **Kabul muayene planı:** Müşteri ürünü kabul ediyor mu, etmiyor mu? Bunun nasıl ortaya konacağını ve nasıl geçerleneceğini tanımlar.

Kısacası kaliteyi sağlamak istiyorsanız metriklerinizi tanımlamalı, ölçümlerinizi yapmalı, süreci sürekli gözden geçirip gerekirse güncellemeli, hem yazılımı hem de bütün süreci test etmelisiniz. Bunların hepsi önceden tanımlı bir plan çerçevesinde yapılmalıdır. En doğru yol, kalite ölçütlerini, planlarını ve gözden geçirmelerini yazılım geliştirme planına eklemektir. Bunların hepsi hem zaman hem de iş yükü olarak projenin toplam maliyetini artırır.

6. Test Aşaması ve Test Türleri

Test, hataları bulmak için yapılır. Hata bulununca, hatanın yapıldığı yere geri dönülüp düzeltilir. Bu yüzden gerçekleştirme yapılır yapılmaz test etmek gerekir.

Hoca sıralamayı önceden özetledi. Üretilen her yazılım parçası birim testlerinden geçer. Parçalar doğru çalışıyorsa bir arada da doğru çalışıp çalışmadıklarına bakılır. Sonra isterleri, yani gereksinimleri doğru biçimde karşılayıp karşılamadıkları, en son da kalite standartlarını karşılayıp karşılamadıkları test edilir. Yazılım pek çok açıdan sınanır. Temelde yapılan testler dört başlıkta toplanır.

6.1 Fonksiyonel (İşlevsel) Testler

Bu testlerde girdi, işlem ve çıktı (input, process, output) sınanır, uç değer analizleri de yapılır. Sisteme doğru, çöp olmayan girdiler verildiğinde sistem doğru çalışıp beklenen sonuçları üretiyor mu diye bakılır.

Sonuç yanlışsa sorun girdide mi, işlemlerde mi, sonuç üretiminde mi diye geri dönülüp bakılır. Hatanın kaynağına göre geri dönülen yer ve maliyet değişir:

- **Kod hatası:** “Yanlış kodlamışız” ya da “veriyi yanlış almışız, yanlış çevirmişiz” denir ve kod düzeltilir.
- **Tasarım hatası:** Kodda hata yoktur, ama algoritma olmamıştır. Tasarıma dönülür, tasarım değiştirilir, kod güncellenir ve yeniden test edilir.
- **Problemin yanlış anlaşılması:** Analize geri dönmek gerekir. Bu, maliyeti doğal olarak çok artırır.

Projeyi küçük parçalara bölüp her parçayı iteratif olarak üretme fikrinin günümüzde bu kadar kabul görmesinin nedeni de budur. Hocanın ifadesiyle proje, adım adım, “yiyebileceğimiz lokmalar hâlinde” ilerler.

6.2 Performans Testleri

Her türlü başarım ölçülür ve bunların yeterli olup olmadığına, kalite beklentilerini karşılayıp karşılamadığına bakılır. Tepki süresi, bellek kullanımı, iletişim ihtiyacı ve iletişim hızı ölçülür. Darboğazlar da belirlenir. Örneğin hacim arttıkça bellek yetersizliğine gidileceği öngörülüyorsa sistemin bellek miktarı artırılmalıdır. İletişim yavaşsa iletişim kanalının hızının artırılması gerektiği söylenebilir.

6.3 Dayanıklılık (Stres) Testleri

Dayanıklılık testleri, performans testinin uç noktalarını kontrol eder. Sisteme aşırı yüklenilir. Hocanın örneğinde sistem, bant genişliğine ve tasarıma göre 1000 eş zamanlı kullanıcı için tasarlanmıştır. İki tür yüklenme yapılabilir:

- 1000 kullanıcı, belli bir süre boyunca, saatlerce sistemi sürekli meşgul eder.
- 1000 yerine 5000 ya da 10.000 kullanıcı yüklenir. Bu, tasarım yükünün 5 ve 10 katıdır (ara adım eklendi).

Sonra şunlara bakılır: Nasıl bir darboğaz oluşuyor? Nereler “patlıyor”, sistemin neresinden “tıs sesi” çıkıyor? İlk hata nerede oluşuyor, hangi hata neyi nasıl tetikliyor? Sistem bunları kaldırabiliyor mu? Girdilerde bir kuyruklama mekanizması kurulmuş mu, ne kadar düzgün

çalışıyor? Ne kadar veri kaybı oluyor? Bu metrikler dayanıklılık testinin sonucunda ortaya çıkar.

Sonuçların bir kısmı kabul edilebilir. “Bizim için 1000 kullanıcı yeter, 10.000 gibi bir projeksiyonumuz yok” denirse test ona göre yapılır. Ama “Normalde 300-500 kullanıcımız olur, ama bazen öyle bir kampanya yapıyoruz ki aynı anda 10.000 kişi bir şey yapmaya çalışıyor” denirse, sistemin tasarımı ölçeklenebilir olacak biçimde değiştirilmelidir.

Soru: Veriler localhost’ta tutulursa, aynı anda bin kullanıcının bağlanma ihtimali sıkıntı olur mu?

Cevap: Hoca bu aşamada öğrencilerden yazılım kalite faktörleri konusunda bir beklentileri olmadığını, stres testi yapılmayacağını söyledi. Önce bir işi yapmayı, işlevsel gereksinimleri karşılayan bir sistem geliştirmeyi öğrenmek gerekir. Bunları da karşılayacak biçimde yapmak daha sonra ilaveten öğrenilir.

> **Hocanın vurgusu:** Ama bunları bilmeniz lazım. Farkında olun. Bunların hepsi ileride başınıza bela olacak.

6.4 Yapısal Testler

Hoca yapısal testlerin fonksiyonel testlere çok benzediğini, hatta onların içinde yer alabileceğini söyledi. Burada sistemin iç yapıları sınanır: Kodlar doğru mu, program parçaları mantıksal olarak düzgün çalışıyor mu? Bütün algoritmalar düzgün çalışıyor mu? Modüllerin içindeki mekanizmalar ve kontrol akışı düzgün mü? Yapısal yazılım testinde mantıksal yollar denetlenir.

> **Ek bilgi (derste anlatılmadı):** Literatürde bu iki tür çoğunlukla birbirini tamamlayan ayrı yaklaşımlar olarak geçer. Fonksiyonel test, kodun içine bakmadan girdi ve çıktıya bakan kara kutu testidir. Yapısal test, kodun iç mantığını ve yollarını denetleyen beyaz kutu testidir. Uç değer analizi ise geçerli girdi aralığının sınırlarındaki değerleri ve hemen yanlarını sınamaktır.

7. Test Adımları: Test Spirali

Hoca test adımlarını süreç adımlarıyla eşleştirmek için bir spiral kullandı. Spiral dışarıdan sistem geliştirmeyle başlar. İçeri girildikçe gereksinimler, tasarım ve kodlama gelir. Şekilde kodlama, başka aşamayla karışmasın diye “K” harfiyle gösterilmişti.

Her aşamanın karşısında bir test adımı bulunur. Testler spiralin içinden dışına doğru ilerler:

Süreç aşaması	Karşılık gelen test	Ne sınanır?
Kodlama	Birim (ünite) testi	Kodlanan her parçanın doğruluğu
Tasarım	Bütünleme testi	Modüllerin ve bileşenlerin birlikte doğru çalışması
Gereksinimler	Onaylama testi	Bütün gereksinimlerin karşılanması ve doğru çalışması
Sistem geliştirme	Sistem testi	Kalite faktörleri

7.1 Birim Testi

Her modül, her parçacık için ayrı ayrı yapılır ve kodun doğruluğunu test eder. Kodlanan her parça mutlaka birim testine tabi tutulur ve böylece geçerlenir.

7.2 Bütünleme Testi

Tasarımın doğru olup olmadığını ölçer: Bu modüller, bu parçalar birlikte doğru çalışıyor mu, sıkıntı var mı? Modüller bağlanarak sistem oluşturulurken bütün olarak ya da artmalı olarak yapılabilir. Sınanan yalnız yazılım parçaları değildir. Veri bileşenleri ve haberleşme bileşenleriyle birlikte çalışma da sınanır. Hata varsa düzeltilir. Belki kodlama da değişir ve birim testleri ile bütünleme testi yeniden yapılır.

7.3 Onaylama Testi

Bütünlemede sorun yoksa bir önceki aşamayı test etmek için onaylama testi yapılır. Daha önce anılan kabul planındaki kabul testi de buraya girer. Bütün gereksinimler sistem tarafından karşılanıyor mu, doğru çalışıyor mu diye bakılır.

Burada bir sorun çıkarsa problem daha da büyür. “Bazı şeyleri yanlış yapmışız” denip yeniden analiz ve tasarım yapılır. Değişikliğin değdiği her yer gözden geçirilir, sonra kodlama, güncelleme, birim testleri, bütünleme testi ve onay testi tekrarlanır. Hoca bunu sürekli gidip gelmek zorunda kalınan, “bir aşağı bir yukarı” bir yapı olarak anlattı.

7.4 Sistem Testi

Müşteri sistemi onayladı, istediklerinin hepsinin yapıldığını söyledi. Bunun ardından sistem geliştirmenin tamamını karşılayacak sistem testleri yapılır. Burada daha çok kalite faktörleri sınanır: performans, dayanıklılık (stres) ve güvenlik testleri bu kapsamdadır. Hata varsa yine düzeltilir.

7.5 Test Güdümlü Geliştirme

Hoca, testlerin ayrıntısının bu dersin kapsamında olmadığını, test mühendisliğinin ayrı bir alt çalışma alanı olduğunu belirtti. Günümüzde “önce testleri yaz, sonra kodu yaz” diyen yaklaşımlar da vardır. Buna test güdümlü geliştirme (test driven development) denir. Biraz daha fazla kod yazmayı gerektirir. Ama pek çok programlama dilinde birim testleri için hazır test kütüphaneleri vardır. Bunların beklentilerini karşılayacak ilave kodlarla geliştirme yapmak yeterlidir. Böylece testler daha otomatik ve daha sürekli yapılır. Hoca birim testi yapmanın da o kadar zor olmadığını ekledi.

8. Yeni Sisteme Geçiş İçin Hazırlık

Testler geçti, düzeltmeler yapıldı, sistem onaylandı. Sistem söylenen işleri yapıyor, işlevsel olmayan beklentileri ve kalite faktörlerini de karşılıyor. Peki “hayırlı olsun” deyip herkes yoluna mı gidecek? Bu sistem nasıl kullanılmaya başlayacak, kim devreye alacak? Sonraki aşama yeni sisteme geçiştir. Hoca bunun için yapılması gerekenleri sıraladı.

8.1 Donanım ve İletişim Altyapısı

Önce donanımlar kurulmalı ve iletişim altyapısı oluşturulmalıdır.

8.2 Eski Kayıtların Aktarılması

Mevcut sistemde kayıtlar varsa yeni sisteme aktarılmaları, belki de değiştirilmeleri gerekir. Hoca bunların çok sancılı süreçler olduğunu söyledi ve öğrencilerin birebir şahit olduğu örneği verdi: USİS’ten OBS’ye geçişte bütün bilgiler taşındı. Bazı öğrenciler “Benim şu dersim gözükmüyor” ya da “Ben AA almıştım, neden böyle gözüküyor?” gibi sorunlar yaşadı. Yani ufak hatalar ortaya çıkabilir.

> **Hocanın vurgusu:** Eski kayıtların aktarılması bazen kendi içinde minik bir proje olarak ele alınması ve işletilmesi gereken ayrı bir çözümdür.

8.3 Yeni Temel Verilerin Girilmesi

Yeni sistemde, eskisinde hiç olmayan temel tanım ve kayıtların girilmesi de gerekebilir. Hocanın örneğinde eski sistem adresleri yalnızca metin olarak tutar. Yeni sistemde ise coğrafi bilgi sistemi entegrasyonu için il, ilçe, mahalle ve semtin yanında coğrafi koordinatlar gibi ilave bilgilere ihtiyaç vardır. Bu veriler atlanmamalıdır.

8.4 Eğitim

> **Hocanın vurgusu:** Eğitim şart. Her türlü geçişte eğitim mutlaka verilmelidir.

Eğitim üç kesime verilir:

- **Sistemle ilgilenecek kişiler:** Sistemin nasıl çalışacağı, bakımının nasıl yapılacağı, temel parametrelerin nasıl girileceği ve değiştirileceği anlatılır.
- **Son kullanıcılar (uç kullanıcılar):** Belirli işlerin nasıl yapılacağı, hangi modüllerin nasıl kullanılacağı anlatılır.
- **Bütün yönetim kademesi:** Raporlamaların ve takiplerin nasıl yapılacağı anlatılır.

Donanım kurulduktan, altyapı oluşturulduktan, veriler aktarıldıktan ve eğitimler verildikten sonra sistemi devreye alma noktasına gelinir.

9. Yeni Sisteme Geçiş Yöntemleri

Bir sistemi devreye almak için toplam beş yöntem önerilmiştir. Bunlara yeni sisteme geçiş yöntemleri denir: doğrudan, paralel, dereceli, dağıtılmış ve prototip geçiş. Hoca her birini eski ve yeni sistemin zaman içindeki kullanımını gösteren bir grafikle anlattı.

9.1 Doğrudan Geçiş

Zamanda bir T noktası vardır. T anından önce hep eski sistem kullanılır. "17 Mayıs 2023 sabah 00.00'dan itibaren yeni sisteme geçiyoruz" denir. O anda yeni sistem devreye alınır, eski sistem kapatılır. Bütün işler artık yalnızca yeni sistemde yapılır.

- **Artısı:** Eski sistem mazide kalmıştır. Onunla ilgilenmek, ondan iş beklemek gerekmez. Her şey yeni sistemde bir kez yapılır.
- **Eksisi:** Testlerde gözden kaçmış noktalar olabilir. Yeni sistem çok kritik bir noktada patlarsa ya da yanlış çalışırsa "gol yeriz".

Belirli sistemlerde doğrudan geçiş kullanılabilir. Ama pek çok kişi, "Bir şey olursa eskiden devam ederiz, yeniye sonra bakarız" diyebilmek için eski çalışma düzeninin yedek gibi bir süre devam etmesini isteyebilir. Paralel geçiş tam bu noktada gündeme gelir.

9.2 Paralel Geçiş

Burada iki zaman noktası vardır: T_0 ve T_1. "17 Mayıs sabah 9'dan itibaren yeni sistemi devreye alıyoruz, şu adresten erişebilirsiniz, eğitimlerinizi verdik" denir. Ama her ihtimale karşı, bir hafta ya da bir ay gibi bir süre, yani T_1 anına kadar eski sistem bütün özellikleriyle kullanılmaya devam eder. Yeni sistem de tamamen devrededir.

Sıkıntı şudur: T_0 ile T_1 arasında her iş iki kez yapılır. Bir sipariş hem eski hem yeni sisteme girilir. Raporlar ve tablolar iki sistemde de oluşturulur.

Artısı ise şudur: Yeni sistemde sıkıntılı noktalar fark edilse bile eski sistem çalışmaya devam ettiği için hizmetler aksamaz. Gerekirse T_1 biraz ötelenir, bu sürede yeni sistemde güncellemeler yapılır. T_1 anında "İkna olduk, yeni sistem çok iyi çalışıyor, eksiği yok" denince eski sistemin "şalteri indirilir". T_1'den sonra her iş bir kez, yeni sistemde yapılır.

Hoca, özellikle görev açısından kritik (mission critical) işler varsa eski sistemin bir süre daha gerçekten işlevsel olarak yaşatılmasının tercih edilebileceğini söyledi.

9.3 Dereceli Geçiş

Yine T_0 ve T_1 noktaları vardır, ama bu kez geçiş işler üzerinden kademeli yapılır: "Sipariş için şu tarihten itibaren yeni sistemi kullanacağız, eski sisteme sipariş girmeyin." Sonra başka modüller eklenir. Geçiş modül modül, özellik özellik, servis servis ilerler. En basitten başlayıp en karmaşığa doğru gitmekte fayda vardır.

Güzelliği şudur: Herhangi bir anda bir iş ya eskide ya yenide yapılır, ikisinde birden yapılmaz. Bu yüzden paralel geçişteki gibi iki kat iş yükü olmaz.

Bedeli ise eski ve yeni sistemin sürekli entegre olarak veri alışverişi yapabilmesidir. Siparişler yeni sistemde yapılırken depo, stok, üretim ve sevkiyat eski sistemde devam eder. Sonra depo, üretim, kargo sırayla alınır. T_1'e gelindiğinde eski sistemden kullanılan hiçbir şey kalmaz. Eski sistem, hocanın ifadesiyle yavaş yavaş "fadeaway" olur.

> **Hocanın vurgusu:** Bu entegrasyonu yapamıyorsanız dereceli geçemezsiniz. Paralel geçişi tercih etmeniz gerekir.

Hata çıkarsa o modül için eski sisteme dönülebilir ve yenisi bitene kadar eskisi kullanılabilir. Hoca dereceli geçişi paralel geçişin biraz yumuşatılmışı olarak tanımladı: Aynı işi iki kez yapmayalım, bazı işleri yenide, kalanları eskide yapalım.

9.4 Dağıtılmış Geçiş

Grafikte bir merdiven görülür. Hoca bunun dereceli geçişe çok benzediğini söyledi. "Arkadaşlar, insan kaynakları yeni sistemi kullanacak, geri kalan herkes eski sistemde." Sonra "İnsan kaynaklarının yanına depoyu da ekledik, depo eski sistemi artık kullanmayacak." Yani işletmenin parçaları ayrı ayrı yeni sisteme geçer.

Hocanın benzetmesi yine öğrenci bilgi sistemiydi: Ağustos başında lisans öğrencileri OBS'ye geçer, yüksek lisans eski sistemde devam eder. Sonra yüksek lisans, sonra doktora geçer. Ardından özlük bilgileri ve lisansüstü sınavları taşınır.

Soru: Dereceli geçişle dağıtılmış geçişin farkı nedir?

Cevap: Dereceli geçişte özellikler, dağıtılmış geçişte departmanlar yavaş yavaş geçer. Aradaki fark budur.

9.5 Prototip Geçiş

Prototip geçiş paralel geçişe benzer, ama başlangıcı farklıdır. Eski sistem hayatına devam eder ve T_1 anına kadar bütün işler onunla yapılır. T_0 anında yeni sistemin geliştirilmesi başlatılır: En az özellikle bir sürüm 0 çekirdeği, ilk prototip üretilir. Günlük işler eski sistemde sürerken prototip yavaş yavaş ilerler. İş yükü başta azdır, ama zamanla iki sistemde de aynı işleri yapar hâle gelinir.

Prototip T_1'e kadar öyle bir olgunlaşır ki bütün beklentileri ve kalite standartlarını karşılar. Bu noktada "Eski sistemin fişini çekin" dersiniz. Prototip artık prototip değildir, yeni sistem olarak yoluna devam eder. Hoca bunu "oyuncu değişikliği" gibi bir geçiş olarak nitelendirdi.

Farkı şudur: Prototip başarısız olursa, hiç yokmuş gibi eski sistemle yola devam edilir. Eski sistem istenildiği kadar uzun yaşayabilir. Prototip bu sırada geliştirilir, kullanılır, kullanımı artar. Yeni özellikleriyle birlikte ihtiyaçları karşıladığında geçiş yapılır.

> **Ders sonrası not:** Hoca prototip geçişi "dördüncü ve sonuncu" diye andı. Doğrusu beşincidir. Hocanın başta söylediği gibi toplam beş yöntem vardır.

9.6 Yöntemlerin Karşılaştırılması

Hoca beş yöntemi eski sistemin kullanımına göre özetledi. Prototip ve paralel geçişte eski sistem bir süre tam olarak kullanılır. Dereceli ve dağıtılmış geçişte azalarak kullanılır. Doğrudan geçişte "küt diye" bir anda geçilir.

Yöntem	Eski sistemin kullanımı	Artı	Eksi
Doğrudan	T anında biter	İş bir kez yapılır, eski sistemle uğraşılmaz	Risk yüksektir, gözden kaçan hata kritik işleri etkiler
Paralel	T_1'e kadar tam	Hizmet aksamaz, T_1 ötelenebilir	T_0–T_1 arasında her iş iki kez yapılır
Dereceli	Özellik özellik azalır	Bir iş tek sistemde yapılır, iki kat iş yükü olmaz	İki sistem sürekli entegre olmalıdır
Dağıtılmış	Departman departman azalır	Hoca dereceliye çok benzediğini söyledi	Hoca ayrıca belirtmedi
Prototip	T_1'e kadar tam	Prototip olmazsa eski sistemle devam edilir	İş yükü zamanla iki sisteme yayılır

Hocaya göre yöntemlerin birinde iş yükü, birinde risk artar. Projeye, sektöre, alana ve bilgi sistemine göre uygun geçiş seçilir. Bu seçim sona bırakılmaz. Fizibilite aşamasında, hatta proje planı oluşturulurken, project charter'da bile tartışılıp ortaya konması gereken bir konudur.

10. Bakım

Yeni sistem devreye alındı ve işlerin hepsi onunla yapılıyor. Artık o "yeni" değil, canlı bilgi sisteminizdir. Ama hocaya göre bir yazılım ekosistemi, yaşayan bir organizma gibidir: Sürekli evrimleşir ve değişir. Testler yapılmış olsa da her zaman yazılımsal, donanımsal ya da kullanıma bağlı hata oluşabilir. İhtiyaçlar değişebilir, eskiden olmayan yeni işler ortaya çıkabilir. Bu değişiklikler bakım çalışmasıyla yapılır.

10.1 Bakım Türleri

İyileştirici bakım: Yeni özellik eklemek ya da mevcut işlevleri daha iyi veya değişen ihtiyaçlara uygun çalışacak biçimde değiştirmektir. Hocanın örneği üniversitedeki not sistemidir. Harf notlarının aralıkları ve skalası değişti, daha çok parçalı bir sisteme geçildi. Bu yüzden notların tutuluş biçimi ve ağırlıklı genel not ortalaması hesabı değişmek zorunda kaldı.

Uyarlayıcı bakım: Altyapısal değişimlere uyum sağlamaktır. Teknoloji eskiden sunucu dünyasındadır, sonra küme (cluster) dünyasına geçilir, sonra bulut çıkar: "At buluta unut." Aynı bilgi sistemi, aynı uygulama, aynı mantık artık bambaşka bir altyapıda çalışacaktır. Yeni duruma bir uyarlama gerekir.

Düzeltici bakım: Testler ne kadar yapılmış olursa olsun "hatasız kul olmaz, hatasız yazılım da olmaz". Testlerde fark edilmeyen, sonradan ortaya çıkan hataların düzeltilmesidir.

Demek ki bir yazılımın kullanım ömrü boyunca üç çeşit bakım yapılır: Değişikliklere göre iyileştirici, altyapıya uyum için uyarlayıcı, üründeki hatalar için düzeltici bakım.

> **Ek bilgi (derste anlatılmadı):** Literatürde bu üç türe çoğunlukla dördüncü olarak önleyici (preventive) bakım da eklenir. Önleyici bakım, gelecekteki sorunları önlemek için kodun yeniden düzenlenmesidir.

10.2 Bakımın Yaşam Ömründeki Payı ve USİS Örneği

> **Hocanın vurgusu:** Bir bilgi sisteminin bütün yaşam ömrünün en az %40, en çok %80 kadarı bakım evresidir. Yazılım ne kadar kullanılırsa bakım da o kadar sürer.

Hoca bunu üniversitenin öğrenci bilgi sistemleriyle örnekledi. OBS, 2022 yazında geçiş yaşanarak devreye alındı. Geçiş, kendi anlattığı yöntemlere de güzel bir örnektir. Doğrudan geçiş yapıldığı söylendi, ama USİS bir süre aynen çalışmaya devam etti, bu yüzden aslında paralel geçiş yaşandı. Bir öğrenci de bu dönemde USİS'e girebildiğini hatırlattı. Aynı zamanda dağıtılmış geçiş de vardı: Önce lisans, sonra yüksek lisans, sonra yüksek lisansın farklı parçaları geçti. Hocaya göre paralel ya da dağıtılmış geçişler sansasyonel değildir. "Devrim yaptık, doğrudan geçiyoruz" demek reklam için daha iyidir. Sonuçta paralel ve dağıtılmış geçiş bir arada yaşanmıştır.

USİS için hesap adım adım şöyledir:

- **Verilen:** USİS bu üniversitede ilk kez 2003'te kullanılmaya başlandı. Hoca devreden kalkış için önce 2021 dedi, ardından "2021, 22 hatta" diye ekledi ve süreyi 18 yıl olarak aldı. Devreye almadan önceki analiz, tasarım, test ve gerçekleştirme yaklaşık 1 yıl sürdü. Daha önceki planlama bu süreye dâhil değildir. Devreye alındıktan sonra geliştirme sürdü. Sistem, 2004-2005 gibi olgun biçimde çalışır hâle geldi. Hoca bu dönemi 2 yıllık prototip dönemi saydı.
- **Kullanım süresi:** $2021 - 2003 = 18$ yıl.
- **Bakım süresi:** $18 - 2 = 16$ yıl.
- **Toplam süre:** $1 + 2 + 16 = 19$ yıl.
- **Sonuç:** Hocanın ifadesiyle 19 yılın 16'sı bakımda geçti.

Yani bir bilgi sistemi 16 yıl hizmet ediyorsa 16 yıl da bakımı yapılıyor demektir. Bu süreçte pek çok ihtiyaç ve özellik değişti, çok fazla iyileştirici bakım yapıldı. Artan hacmi ve istekleri

karşılamak için çok fazla altyapı değişimi, yani uyarlama yapıldı. Hocaya göre bu uyarlamalar doğru biçimde yapılmadı. Az da olsa hatalar düzeltildi. İyileştirici bakımların ardından da pek çok düzeltici bakım gerekti.

> **Ders sonrası not:** Hoca hem "18 sene" hem "19 sene" dedi. İkisi birbiriyle çelişmez: 18 yıl devreye alındıktan sonraki kullanım, $19 = 18 + 1$ ise öncesindeki geliştirme yılıyla birlikte toplamdır. Bu örnekte bakımın payı $16/19 \approx \%84{,}2$ çıkar. Bu oran, hocanın verdiği %40-%80 aralığının üstündedir. Bitiş 2022 alınırsa kullanım 19, bakım 17, toplam 20 yıl olur ve pay $17/20 = \%85$ çıkar. Sonuç yine aralığın üstündedir. Örnek, bakımın en uzun evre olduğunu gösterir, ama aralığın içinde kalmaz.

Hoca, özellikle bir işletme için temel olan köklü bilgi sistemlerinde değişikliğin çok fazla işlenmediğini ekledi. Çünkü ne kadar değişiklik, o kadar risk, hata olasılığı ve tekrar iş yapma olasılığı demektir. Bu yüzden mümkün olduğunca uzun ömürlü ve değiştirilebilir sistemler tasarlamaya gayret edilir.

11. Döngünün Yeniden Başlaması ve Günümüz Yaklaşımları

Bir gün sistem ihtiyaçları karşılamaz olur ya da istekler farklılaşır. "Yeni bilgi sistemine geçelim" denir ve süreç yeniden başlar: ön inceleme, fizibilite, analiz, tasarım, geliştirme, test, devreye alma, bakım. Yani süreç, uzun dönemlerde de olsa bir devinim hâlinde kendini sürdürür.

Günümüzde bu iş daha otomatik hâle getirilerek sürekli geliştirme ve sürekli entegrasyon süreçlerine geçiliyor. Yazılım üretimi bir operasyonel süreç yönetimi olarak yapılmaya çalışılıyor. Buna DevOps ve CI/CD süreçleri deniyor. Bunun için yazılım tasarımı daha modüler, daha ufak ve granüler parçalar hâlinde yapılıyor.

Bu da yeni problemler getirir: Parçalar nasıl eş zamanlanacak, nasıl haberleşecek? Hata olduğunda işlem nasıl geri alınacak? Performans nasıl ayarlanacak, dağıtım nasıl yapılacak? Hocaya göre bunlar bugün özellikle yazılım mühendislerinin en çok kafa yorduğu noktalardır. Günümüzde mikro servis mimarisi, hatta sunucusuz mimariler kullanılıyor, mikro servislerden nano servislere doğru gidiliyor.

Ama bu bir ödünleşimdir (trade-off). Parçalar ufaldıkça onları bir araya getirmek zorlaşır. Parçalar büyüdükçe yapı hantallaşır. Hocanın benzetmesi Lego'dur. Ufak çocuk Legosu ile teknik Lego bir değildir. Birinde binlerce parçayla çok iyi bir şey yaparsınız, ama yapana kadar akla karayı seçersiniz. Öbüründe üç parçayı koyup hemen bitirirsiniz, ama sonuç köşeli, "Minecraft gibi" olur. Yazılım mühendisleri ve yazılım mimarları gerçek hayatta bu problemleri bu araçlarla çözmeye çalışır.

Dersin Toparlaması

Bu derste, tasarımdan sonra bir bilgi sisteminin ayağa kalkıp yaşamını sürdürene kadar geçtiği aşamalar görüldü. Kodlama, dile özgü standartlara uyan, okunabilir, düzenlenebilir, test edilebilir, açıklamalı ve anlamlı isimli kod yazmayı gerektirir. Dil seçimi bir araç seçimidir ve mühendisin sorumluluğundadır. On bir kalite faktörü işlevsel olmayan gereksinimlerle yakından ilişkilidir, mimariyi baştan değiştirebilir ve ciddi maliyet getirir. Bu yüzden en başta netleştirilmelidir. Kalite, metrikler, gözden geçirmeler ve planlı testlerle izlenir. Test spiralinde birim, bütünleme, onaylama ve sistem testleri sırasıyla kodlamayı, tasarımı, gereksinimleri ve sistemin bütününü sınar.

Geçiş için altyapı, veri aktarımı ve eğitim hazırlanır, ardından projeye uygun beş yöntemden biri seçilir. Geçiş bitince sistemin en uzun evresi olan bakım başlar. Bakım iyileştirici, uyarlayıcı ve düzeltici olarak yürür ve sistem ihtiyaçları karşılayamaz olduğunda döngü başa döner. Bugünün DevOps, CI/CD ve mikro servis yaklaşımları bu döngüyü küçük parçalar hâlinde sürekli işletmeye çalışır. Hoca son olarak, öğrencilerin kalan haftalardaki sunumlarda aynı ya da benzer problemlerin ne kadar farklı yollarla çözülebildiğini göreceğini söyledi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.